

实验五：SLR 引导的语义分析框架实现

姓名：李祥熙 学号：2234412799

2026 年 5 月 30 日

目录

1	实验目的	2
2	实验过程	2
2.1	实验环境	2
2.2	输入与预处理	2
2.3	实现步骤	2
3	实验内容	3
3.1	任务一：SLR 引导的语义分析框架	3
3.2	任务二：AST 构建与语义检查	3
4	实验结果	4
4.1	编译与运行	4
4.2	AST 与语义输出	5
4.3	符号表结果汇总	6
5	总结	7

1 实验目的

本实验的目标如下：

- 目标一：在 SLR(1) 语法分析基础上集成语义动作并维护属性栈
- 目标二：构建抽象语法树 (AST) 并输出可视化结构
- 目标三：维护符号表与作用域，完成类型检查与语义错误检测

2 实验过程

2.1 实验环境

- 操作系统：Linux
- 编程语言：C
- 编译器：GCC
- 其他关键工具：终端与文本编辑器

2.2 输入与预处理

输入包括两部分：

- SLR(1) 分析表：由实验四的 LR(0) 规范族与 FOLLOW 集生成（在本实验代码中自动构造）。
- Token 流：来自实验二扫描器输出（如 `tokens.out`），包含类型与词素。

预处理的关键步骤：

- 将 Token 类型映射到文法终结符（例如 `ID -> id`、`INT/# -> num`）。
- 为文法自动增广起始产生式，构造 LR(0) 项目集和 SLR(1) ACTION/GOTO 表。

2.3 实现步骤

- 文法扩展与语义动作设计：为函数、语句、表达式等产生式设计对应语义动作。
- 属性栈与语法栈联动：在规约时弹出右部属性并构造父节点/类型信息。
- 符号表与作用域维护：进入/离开代码块时更新作用域，记录重复声明与未声明错误。
- 输出与验证：打印 AST、符号表和语义错误列表。

3 实验内容

3.1 任务一：SLR 引导的语义分析框架

本实验采用“SLR(1) 分析 + 属性栈”结构：语法分析使用 ACTION/GOTO 表驱动，每次规约触发语义动作，构建 AST 并传递类型信息。文法包含函数定义、参数、声明、赋值、条件、循环与表达式。

文法片段（节选）：

```
Func -> void id ( Params ) Block
Decl -> Type DeclList ;
Assign -> id = Expr ;
IfStmt -> if ( Cond ) Block else Block
Cond -> Expr rop Expr
```

说明：该片段展示了本实验支持的关键语句结构，包含函数、声明、赋值与控制流，便于与 Token 流对齐。

3.2 任务二：AST 构建与语义检查

核心语义动作包括：

- 声明语句：遍历声明列表，将变量加入符号表，检查重复声明。
- 赋值语句：检查左值是否声明，比较左右值类型并报告不匹配。
- 条件表达式：使用关系运算符生成条件节点，类型统一为整型逻辑值。

代码片段（声明与赋值语义动作节选）：

```
case 19: /* Decl -> Type DeclList ; */
    result.node = ast_create("decl", value_type_name(rhs_attrs[0].type)
    );
    ast_add_child(result.node, rhs_attrs[1].node);
    declare_decl_list(rhs_attrs[1].node, rhs_attrs[0].type);
    break;

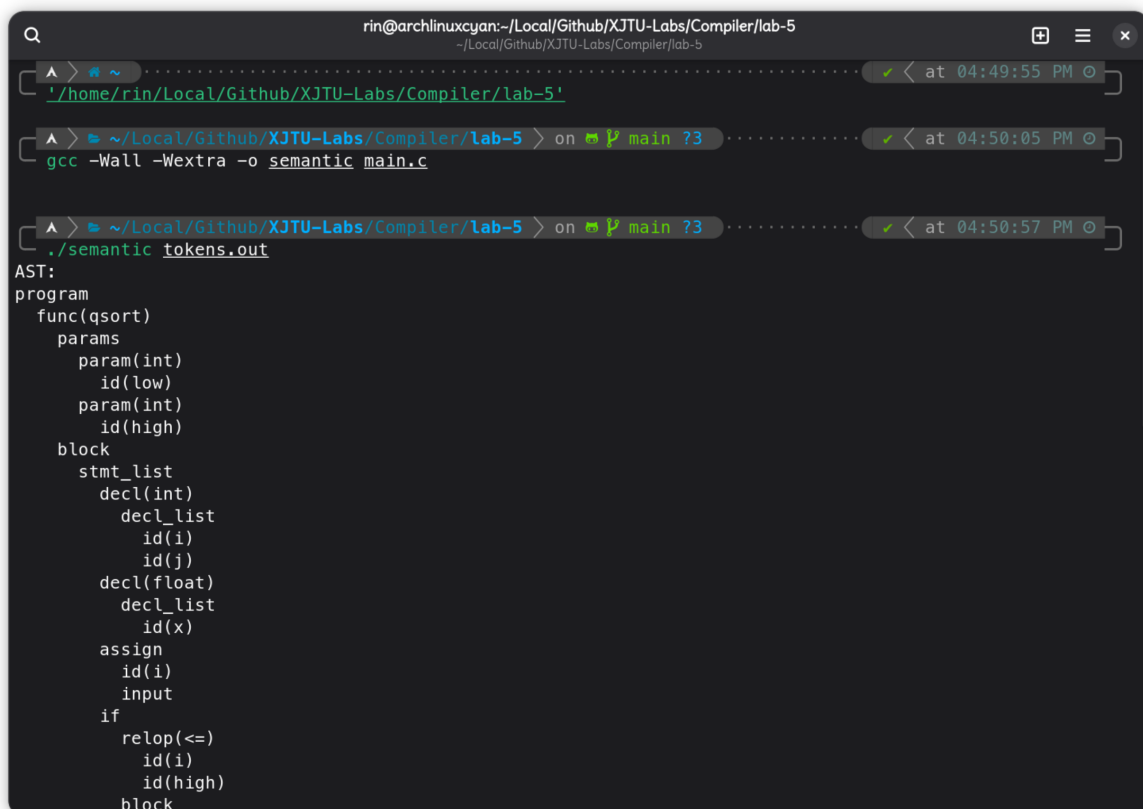
case 22: /* Assign -> id = Expr ; */
    {
        const char *name = rhs_attrs[0].lexeme;
        ValueType lhs_type = TYPE_UNKNOWN;
        if (!lookup_symbol(name, &lhs_type)) {
            record_error("Undeclared identifier");
            lhs_type = TYPE_ERROR;
        }
    }
}
```

```
    }  
    if (lhs_type == TYPE_INT && rhs_attrs[2].type == TYPE_FLOAT) {  
        record_error("Type mismatch");  
    }  
    result.node = ast_create("assign", NULL);  
    ast_add_child(result.node, rhs_attrs[0].node);  
    ast_add_child(result.node, rhs_attrs[2].node);  
}  
break;
```

说明：声明语义动作将标识符加入符号表；赋值语义动作检查类型一致性并构建“assign”AST 节点。

4 实验结果

4.1 编译与运行



```
rin@archlinuxcyan:~/Local/Github/XJTU-Labs/Compiler/lab-5  
~/Local/Github/XJTU-Labs/Compiler/lab-5  
└─> /home/rin/Local/Github/XJTU-Labs/Compiler/lab-5  
└─> gcc -Wall -Wextra -o semantic main.c  
└─> ./semantic tokens.out  
AST:  
program  
  func(qsort)  
    params  
      param(int)  
        id(low)  
      param(int)  
        id(high)  
    block  
      stmt_list  
        decl(int)  
          decl_list  
            id(i)  
            id(j)  
          decl(float)  
            decl_list  
              id(x)  
        assign  
          id(i)  
          input  
        if  
          relop(<=)  
            id(i)  
            id(high)  
          block
```

图 1: 编译与运行示例

说明：图 1 展示了在实验目录下编译并运行语义分析程序的过程，验证了程序能够在给定 Token 流上完成语法与语义分析。

4.2 AST 与语义输出

以下为实际运行输出（节选）：

AST:

```
program
func(qsort)
  params
    param(int)
      id(low)
    param(int)
      id(high)
  block
    stmt_list
      decl(int)
        decl_list
          id(i)
          id(j)
        decl(float)
          decl_list
            id(x)
      assign
        id(i)
        input
      if
        relop(<=)
          id(i)
          id(high)
        block
          stmt_list
            assign
              id(x)
              num(-3.14)
            print
              id(i)
          block
            stmt_list
              while
                relop(<)
```

```

    id(j)
    num(10)
block
  stmt_list
    assign
      id(j)
      add
        id(j)
        num(1)

```

Symbol Table:

Name	Type	Scope
low	int	0
high	int	0
i	int	1
j	int	1
x	float	1

Semantic Errors:

None

说明：AST 显示了函数“qsort”的参数、块结构、条件与循环嵌套关系；符号表记录了参数与局部变量的类型与作用域；语义错误列表为空，说明该输入在本规则下通过了检查。

4.3 符号表结果汇总

表 1: 符号表条目（节选）

标识符	类型	作用域
low	int	0
high	int	0
i	int	1
j	int	1
x	float	1

说明：该表格对应符号表输出，作用域 0 表示函数参数，作用域 1 表示函数体内部的局部变量。

5 总结

本实验成功完成了 SLR 引导的语义分析框架：

- 基于 SLR(1) 分析表完成语法驱动的语义动作执行。
- 构建了表达式与语句层面的 AST，支持函数、控制流与块结构。
- 实现符号表与作用域管理，支持重复声明、未声明与类型不匹配检查。

后续可扩展：增加函数返回类型检查、参数数量/类型匹配与更丰富的语句类型。

附录：实验源代码

A.1 主要源码：main.c

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE 1024
#define MAX_LINES 256
#define MAX_PRODS 256
#define MAX_RHS 8
#define MAX_SYMBOLS 128
#define MAX_STATES 256
#define MAX_ITEMS 2048
#define MAX_NAME 64
#define MAX_TOKENS 2048
#define MAX_AST_CHILDREN 8
#define MAX_ERRORS 256

typedef struct
{
    int lhs;
    int rhs[MAX_RHS];
    int rhs_len;
} Production;

typedef struct
{
```

```
    int prod;
    int dot;
} Item;

typedef struct
{
    Item items[MAX_ITEMS];
    int count;
} ItemSet;

typedef enum
{
    ACT_NONE = 0,
    ACT_SHIFT,
    ACT_REDUCE,
    ACT_ACCEPT
} ActionType;

typedef struct
{
    ActionType type;
    int value;
} ActionCell;

typedef enum
{
    TYPE_UNKNOWN = 0,
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_ERROR
} ValueType;

typedef struct ASTNode
{
    char label[MAX_NAME];
    char lexeme[MAX_NAME];
    struct ASTNode *children[MAX_AST_CHILDREN];
    int child_count;
} ASTNode;

typedef struct
```



```
{
    ASTNode *node;
    ValueType type;
    char lexeme[MAX_NAME];
} Attribute;

typedef struct
{
    char name[MAX_NAME];
    ValueType type;
    int scope;
} SymbolEntry;

typedef struct
{
    int symbol;
    char lexeme[MAX_NAME];
} TokenItem;

static int declare_symbol (const char *name, ValueType type);
static void record_error (const char *msg);

static void
trim (char *s)
{
    size_t n = strlen (s);
    while (n > 0 && (s[n - 1] == '\n' || s[n - 1] == '\r'))
    {
        s[n - 1] = '\0';
        --n;
    }
    size_t i = 0;
    while (s[i] && isspace ((unsigned char)s[i]))
    {
        ++i;
    }
    if (i > 0)
    {
        memmove (s, s + i, strlen (s + i) + 1);
    }
    n = strlen (s);
```

```
while (n > 0 && isspace ((unsigned char)s[n - 1]))
{
    s[n - 1] = '\0';
    --n;
}

static int
is_blank (const char *s)
{
    while (*s)
    {
        if (!isspace ((unsigned char)*s))
        {
            return 0;
        }
        ++s;
    }
    return 1;
}

static int
starts_with (const char *s, const char *prefix)
{
    return strncmp (s, prefix, strlen (prefix)) == 0;
}

static int
find_arrow (const char *s)
{
    const char *p = strstr (s, "->");
    if (!p)
    {
        return -1;
    }
    return (int)(p - s);
}

static int
name_index (char names[][MAX_NAME], int count, const char *name)
{

```

```
for (int i = 0; i < count; ++i)
{
    if (strcmp (names[i], name) == 0)
    {
        return i;
    }
}
return -1;
}

static int
add_name (char names[][MAX_NAME], int *count, const char *name)
{
    int idx = name_index (names, *count, name);
    if (idx >= 0)
    {
        return idx;
    }
    if (*count >= MAX_SYMBOLS)
    {
        return -1;
    }
    strncpy (names[*count], name, MAX_NAME - 1);
    names[*count][MAX_NAME - 1] = '\0';
    (*count)++;
    return (*count) - 1;
}

static int
is_nonterminal (char nonterms[][MAX_NAME], int nonterm_count, const
    char *s)
{
    return name_index (nonterms, nonterm_count, s) >= 0;
}

static int
item_equal (const Item *a, const Item *b)
{
    return a->prod == b->prod && a->dot == b->dot;
}
```

```
static int
itemset_contains (const ItemSet *set, const Item *it)
{
    for (int i = 0; i < set->count; ++i)
    {
        if (item_equal (&set->items[i], it))
        {
            return 1;
        }
    }
    return 0;
}

static void
itemset_add (ItemSet *set, const Item *it)
{
    if (!itemset_contains (set, it))
    {
        if (set->count < MAX_ITEMS)
        {
            set->items[set->count++] = *it;
        }
    }
}

static int
item_cmp (const void *a, const void *b)
{
    const Item *ia = (const Item *)a;
    const Item *ib = (const Item *)b;
    if (ia->prod != ib->prod)
    {
        return ia->prod - ib->prod;
    }
    return ia->dot - ib->dot;
}

static void
itemset_sort (ItemSet *set)
{
    qsort (set->items, (size_t)set->count, sizeof (Item), item_cmp);
}
```

```
}

static int
itemset_equal (const ItemSet *a, const ItemSet *b)
{
    if (a->count != b->count)
    {
        return 0;
    }
    for (int i = 0; i < a->count; ++i)
    {
        if (!item_equal (&a->items[i], &b->items[i]))
        {
            return 0;
        }
    }
    return 1;
}

static void
closure (ItemSet *set, const Production *prods, const int *
        nonterm_first,
        const int *nonterm_count, const int *symbol_is_nonterm)
{
    int changed = 1;
    while (changed)
    {
        changed = 0;
        for (int i = 0; i < set->count; ++i)
        {
            Item it = set->items[i];
            const Production *p = &prods[it.prod];
            if (it.dot >= p->rhs_len)
            {
                continue;
            }
            int sym = p->rhs[it.dot];
            if (!symbol_is_nonterm[sym])
            {
                continue;
            }
        }
    }
}
```

```

    int nt_index = sym;
    int start = nonterm_first[nt_index];
    int cnt = nonterm_count[nt_index];
    if (start < 0 || cnt <= 0)
    {
        continue;
    }
    for (int k = 0; k < cnt; ++k)
    {
        Item new_item;
        new_item.prod = start + k;
        new_item.dot = 0;
        if (!itemset_contains (set, &new_item))
        {
            itemset_add (set, &new_item);
            changed = 1;
        }
    }
}

}

static void
goto_set (const ItemSet *src, int symbol, ItemSet *dst,
          const Production *prods, const int *nonterm_first,
          const int *nonterm_count, const int *symbol_is_nonterm)
{
    dst->count = 0;
    for (int i = 0; i < src->count; ++i)
    {
        Item it = src->items[i];
        const Production *p = &prods[it.prod];
        if (it.dot < p->rhs_len && p->rhs[it.dot] == symbol)
        {
            Item moved;
            moved.prod = it.prod;
            moved.dot = it.dot + 1;
            itemset_add (dst, &moved);
        }
    }
}

closure (dst, prods, nonterm_first, nonterm_count, symbol_is_nonterm)

```

```
        ;
    itemset_sort (dst);
}

static int
parse_grammar_lines (char lines[][MAX_LINE], int line_count,
                     char nonterms[][MAX_NAME], int *nonterm_count,
                     char symbols[][MAX_NAME], int *symbol_count,
                     int *start_symbol, Production *prods, int *
                     prod_count)
{
    *nonterm_count = 0;
    *symbol_count = 0;
    *prod_count = 0;

    for (int i = 0; i < line_count; ++i)
    {
        char buf[MAX_LINE];
        strncpy (buf, lines[i], sizeof (buf) - 1);
        buf[sizeof (buf) - 1] = '\0';
        trim (buf);
        if (is_blank (buf) || starts_with (buf, "#") || starts_with (buf,
            "//"))
        {
            continue;
        }
        int arrow = find_arrow (buf);
        if (arrow < 0)
        {
            return 0;
        }
        buf[arrow] = '\0';
        buf[arrow + 1] = '\0';
        char lhs[MAX_NAME];
        strncpy (lhs, buf, sizeof (lhs) - 1);
        lhs[sizeof (lhs) - 1] = '\0';
        trim (lhs);
        if (lhs[0] == '\0')
        {
            return 0;
        }
    }
}
```

```
    add_name (nonterms, nonterm_count, lhs);
}

if (*nonterm_count == 0)
{
    return 0;
}

for (int i = 0; i < *nonterm_count; ++i)
{
    add_name (symbols, symbol_count, nonterms[i]);
}

*start_symbol = 0;

for (int i = 0; i < line_count; ++i)
{
    char buf[MAX_LINE];
    strncpy (buf, lines[i], sizeof (buf) - 1);
    buf[sizeof (buf) - 1] = '\0';
    trim (buf);
    if (is_blank (buf) || starts_with (buf, "#") || starts_with (buf,
        "//"))
    {
        continue;
    }

    int arrow = find_arrow (buf);
    if (arrow < 0)
    {
        return 0;
    }

    buf[arrow] = '\0';
    char rhs_part[MAX_LINE];
    strncpy (rhs_part, buf + arrow + 2, sizeof (rhs_part) - 1);
    rhs_part[sizeof (rhs_part) - 1] = '\0';

    char lhs[MAX_NAME];
    strncpy (lhs, buf, sizeof (lhs) - 1);
    lhs[sizeof (lhs) - 1] = '\0';
```



```
trim (lhs);
if (lhs[0] == '\0')
{
    return 0;
}

int lhs_sym = name_index (symbols, *symbol_count, lhs);
if (lhs_sym < 0)
{
    return 0;
}

char *alt = rhs_part;
while (alt)
{
    char *bar = strchr (alt, '|');
    if (bar)
    {
        *bar = '\0';
    }
    char alt_buf[MAX_LINE];
    strncpy (alt_buf, alt, sizeof (alt_buf) - 1);
    alt_buf[sizeof (alt_buf) - 1] = '\0';
    trim (alt_buf);

    Production p;
    p.lhs = lhs_sym;
    p.rhs_len = 0;

    if (alt_buf[0] == '\0' || strcmp (alt_buf, "epsilon") == 0
        || strcmp (alt_buf, "eps") == 0)
    {
        p.rhs_len = 0;
    }
    else
    {
        char *tok = strtok (alt_buf, " \t");
        while (tok)
        {
            int sym;
            if (is_nonterminal (nonterms, *nonterm_count, tok))
```

```
        {
            sym = name_index (symbols, *symbol_count, tok);
        }
    else
    {
        sym = name_index (symbols, *symbol_count, tok);
        if (sym < 0)
        {
            sym = add_name (symbols, symbol_count, tok);
        }
    }
    if (sym < 0)
    {
        return 0;
    }
    if (p.rhs_len >= MAX_RHS)
    {
        return 0;
    }
    p.rhs[p.rhs_len++] = sym;
    tok = strtok (NULL, " \t");
}

if (*prod_count >= MAX_PRODS)
{
    return 0;
}
prods[(*prod_count)++] = p;

if (bar)
{
    alt = bar + 1;
}
else
{
    alt = NULL;
}
}
}
```

```
    return 1;
}

static int
add_to_set (unsigned char *set, int row, int col)
{
    unsigned char *cell = &set[row * MAX_SYMBOLS + col];
    if (*cell)
    {
        return 0;
    }
    *cell = 1;
    return 1;
}

static void
compute_first (const Production *prods, int prod_count, int
               symbol_count,
               const int *symbol_is_nonterm, unsigned char *first,
               unsigned char *first_eps)
{
    for (int i = 0; i < symbol_count; ++i)
    {
        first_eps[i] = 0;
        if (!symbol_is_nonterm[i])
        {
            add_to_set (first, i, i);
        }
    }

    int changed = 1;
    while (changed)
    {
        changed = 0;
        for (int i = 0; i < prod_count; ++i)
        {
            const Production *p = &prods[i];
            if (p->rhs_len == 0)
            {
                if (!first_eps[p->lhs])
                {
```

```

        first_eps[p->lhs] = 1;
        changed = 1;
    }
    continue;
}

int all_eps = 1;
for (int j = 0; j < p->rhs_len; ++j)
{
    int sym = p->rhs[j];
    for (int t = 0; t < symbol_count; ++t)
    {
        if (first[sym * MAX_SYMBOLS + t])
        {
            if (add_to_set (first, p->lhs, t))
            {
                changed = 1;
            }
        }
    }
    if (!first_eps[sym])
    {
        all_eps = 0;
        break;
    }
}
if (all_eps && !first_eps[p->lhs])
{
    first_eps[p->lhs] = 1;
    changed = 1;
}
}

}

static void
compute_follow (const Production *prods, int prod_count, int
    symbol_count,
                const int *symbol_is_nonterm, const unsigned char *
    first,
                const unsigned char *first_eps, unsigned char *follow,

```

```

        unsigned char *follow_end, int start_symbol)
{
    memset (follow_end, 0, MAX_SYMBOLS * sizeof (unsigned char));
    follow_end[start_symbol] = 1;

    int changed = 1;
    while (changed)
    {
        changed = 0;
        for (int i = 0; i < prod_count; ++i)
        {
            const Production *p = &prods[i];
            for (int j = 0; j < p->rhs_len; ++j)
            {
                int sym = p->rhs[j];
                if (!symbol_is_nonterm[sym])
                {
                    continue;
                }

                int beta_has_eps = 1;
                for (int k = j + 1; k < p->rhs_len; ++k)
                {
                    int next = p->rhs[k];
                    for (int t = 0; t < symbol_count; ++t)
                    {
                        if (first[next * MAX_SYMBOLS + t])
                        {
                            if (add_to_set (follow, sym, t))
                            {
                                changed = 1;
                            }
                        }
                    }
                }
                if (!first_eps[next])
                {
                    beta_has_eps = 0;
                    break;
                }
            }
        }
    }
}

```

```

        if (j + 1 >= p->rhs_len)
        {
            beta_has_eps = 1;
        }

        if (beta_has_eps)
        {
            for (int t = 0; t < symbol_count; ++t)
            {
                if (follow[p->lhs * MAX_SYMBOLS + t])
                {
                    if (add_to_set (follow, sym, t))
                    {
                        changed = 1;
                    }
                }
            }
            if (follow_end[p->lhs] && !follow_end[sym])
            {
                follow_end[sym] = 1;
                changed = 1;
            }
        }
    }
}

static int
set_action (ActionCell *cell, ActionType type, int value)
{
    if (cell->type != ACT_NONE && (cell->type != type || cell->value !=
        value))
    {
        return 1;
    }
    cell->type = type;
    cell->value = value;
    return 0;
}

```

```
static ASTNode *
ast_create (const char *label, const char *lexeme)
{
    ASTNode *node = (ASTNode *)calloc (1, sizeof (ASTNode));
    if (!node)
    {
        return NULL;
    }
    strncpy (node->label, label, MAX_NAME - 1);
    node->label[MAX_NAME - 1] = '\0';
    if (lexeme)
    {
        strncpy (node->lexeme, lexeme, MAX_NAME - 1);
        node->lexeme[MAX_NAME - 1] = '\0';
    }
    node->child_count = 0;
    return node;
}

static void
ast_add_child (ASTNode *parent, ASTNode *child)
{
    if (!parent || !child)
    {
        return;
    }
    if (parent->child_count < MAX_AST_CHILDREN)
    {
        parent->children[parent->child_count++] = child;
    }
}

static void
declare_decl_list (ASTNode *node, ValueType type)
{
    if (!node)
    {
        return;
    }
    if (strcmp (node->label, "id") == 0)
    {

```

```
    if (!declare_symbol (node->lexeme, type))
    {
        char msg[MAX_LINE];
        snprintf (msg, sizeof (msg), "Duplicate declaration of '%s'",
                  node->lexeme);
        record_error (msg);
    }
    return;
}

for (int i = 0; i < node->child_count; ++i)
{
    declare_decl_list (node->children[i], type);
}
}

static void
print_ast (const ASTNode *node, int indent)
{
    if (!node)
    {
        return;
    }
    for (int i = 0; i < indent; ++i)
    {
        printf ("  ");
    }
    if (node->lexeme[0] != '\0')
    {
        printf ("%s(%s)\n", node->label, node->lexeme);
    }
    else
    {
        printf ("%s\n", node->label);
    }
    for (int i = 0; i < node->child_count; ++i)
    {
        print_ast (node->children[i], indent + 1);
    }
}

static const char *
```



```
value_type_name (ValueType t)
{
    switch (t)
    {
        case TYPE_INT:
            return "int";
        case TYPE_FLOAT:
            return "float";
        case TYPE_ERROR:
            return "error";
        default:
            return "unknown";
    }
}

static SymbolEntry g_symbols[MAX_SYMBOLS];
static int g_symbol_count = 0;
static int g_scope_level = 0;

static void
enter_scope (void)
{
    g_scope_level++;
}

static void
leave_scope (void)
{
    if (g_scope_level > 0)
    {
        g_scope_level--;
    }
}

static int
lookup_symbol (const char *name, ValueType *out_type)
{
    for (int i = g_symbol_count - 1; i >= 0; --i)
    {
        if (g_symbols[i].scope > g_scope_level)
        {
```

```
        continue;
    }
    if (strcmp (g_symbols[i].name, name) == 0)
    {
        if (out_type)
        {
            *out_type = g_symbols[i].type;
        }
        return 1;
    }
}
return 0;
}

static int
declare_symbol (const char *name, ValueType type)
{
    for (int i = 0; i < g_symbol_count; ++i)
    {
        if (g_symbols[i].scope == g_scope_level
            && strcmp (g_symbols[i].name, name) == 0)
        {
            return 0;
        }
    }
    if (g_symbol_count >= MAX_SYMBOLS)
    {
        return 0;
    }
    strncpy (g_symbols[g_symbol_count].name, name, MAX_NAME - 1);
    g_symbols[g_symbol_count].name[MAX_NAME - 1] = '\0';
    g_symbols[g_symbol_count].type = type;
    g_symbols[g_symbol_count].scope = g_scope_level;
    g_symbol_count++;
    return 1;
}

static char g_errors[MAX_ERRORS][MAX_LINE];
static int g_error_count = 0;

static void
```

```
record_error (const char *msg)
{
    if (g_error_count >= MAX_ERRORS)
    {
        return;
    }
    strncpy (g_errors[g_error_count], msg, MAX_LINE - 1);
    g_errors[g_error_count][MAX_LINE - 1] = '\0';
    g_error_count++;
}

static int
parse_token_line (const char *line, char *type, char *lexeme)
{
    const char *l = strchr (line, '(');
    const char *r = strchr (line, ')');
    if (!l || !r || r <= l)
    {
        return 0;
    }
    char buf[MAX_LINE];
    size_t len = (size_t)(r - l - 1);
    if (len >= sizeof (buf))
    {
        return 0;
    }
    strncpy (buf, l + 1, len);
    buf[len] = '\0';
    char *comma = strchr (buf, ',');
    if (!comma)
    {
        return 0;
    }
    *comma = '\0';
    strncpy (type, buf, MAX_NAME - 1);
    type[MAX_NAME - 1] = '\0';
    strncpy (lexeme, comma + 1, MAX_NAME - 1);
    lexeme[MAX_NAME - 1] = '\0';
    trim (type);
    trim (lexeme);
    return 1;
}
```

```
}

static int
load_tokens (const char *path, TokenItem *tokens, int max_tokens,
             char symbols[][MAX_NAME], int symbol_count, int *term_col)
{
    FILE *fp = fopen (path, "r");
    if (!fp)
    {
        fprintf (stderr, "Cannot open token file: %s\n", path);
        return -1;
    }

    int count = 0;
    char line[MAX_LINE];
    while (fgets (line, sizeof (line), fp))
    {
        trim (line);
        if (is_blank (line))
        {
            continue;
        }
        if (starts_with (line, "Scanner finished"))
        {
            break;
        }
        char type[MAX_NAME];
        char lexeme[MAX_NAME];
        if (!parse_token_line (line, type, lexeme))
        {
            continue;
        }

        const char *term = NULL;
        if (strcmp (type, "ID") == 0)
        {
            term = "id";
        }
        else if (strcmp (type, "VOID") == 0)
        {
            term = "void";
        }
    }
}
```

```
    }  
    else if (strcmp (type, "INT") == 0)  
    {  
        if (strcmp (lexeme, "int") == 0)  
        {  
            term = "int";  
        }  
        else  
        {  
            term = "num";  
        }  
    }  
    else if (strcmp (type, "IF") == 0)  
    {  
        term = "if";  
    }  
    else if (strcmp (type, "ELSE") == 0)  
    {  
        term = "else";  
    }  
    else if (strcmp (type, "WHILE") == 0)  
    {  
        term = "while";  
    }  
    else if (strcmp (type, "RETURN") == 0)  
    {  
        term = "return";  
    }  
    else if (strcmp (type, "INPUT") == 0)  
    {  
        term = "input";  
    }  
    else if (strcmp (type, "PRINT") == 0)  
    {  
        term = "print";  
    }  
    else if (strcmp (type, "ROP") == 0)  
    {  
        term = "rop";  
    }  
    else if (strcmp (type, "FLO") == 0)
```

```
{
    term = "num";
}
else if (strcmp (type, "FLOAT") == 0)
{
    term = "float";
}
else if (strcmp (type, "ADD") == 0)
{
    term = "+";
}
else if (strcmp (type, "SUB") == 0)
{
    term = "-";
}
else if (strcmp (type, "MUL") == 0)
{
    term = "*";
}
else if (strcmp (type, "DIV") == 0)
{
    term = "/";
}
else if (strcmp (type, "ASG") == 0)
{
    term = "=";
}
else if (strcmp (type, "LPA") == 0)
{
    term = "(";
}
else if (strcmp (type, "RPA") == 0)
{
    term = ")";
}
else if (strcmp (type, "LBR") == 0)
{
    term = "{";
}
else if (strcmp (type, "RBR") == 0)
{
```

```
        term = "}";
    }
    else if (strcmp (type, "CMA") == 0)
    {
        term = ",";
    }
    else if (strcmp (type, "SCO") == 0)
    {
        term = ";";
    }
    else
    {
        fprintf (stderr, "Unsupported token: %s\n", type);
        fclose (fp);
        return -1;
    }

    int sym = name_index (symbols, symbol_count, term);
    if (sym < 0 || term_col[sym] < 0)
    {
        fprintf (stderr, "Token not in grammar: %s\n", term);
        fclose (fp);
        return -1;
    }

    if (count >= max_tokens)
    {
        fprintf (stderr, "Too many tokens.\n");
        fclose (fp);
        return -1;
    }

    tokens[count].symbol = sym;
    strncpy (tokens[count].lexeme, lexeme, MAX_NAME - 1);
    tokens[count].lexeme[MAX_NAME - 1] = '\\0';
    count++;
}

fclose (fp);
return count;
}
```

[illegible]


```

        "Term -> Term / Factor",
        "Term -> Factor",
        "Factor -> ( Expr )",
        "Factor -> id",
        "Factor -> num",
        "Factor -> input ( )" };

char lines[MAX_LINES][MAX_LINE];
int line_count = (int)(sizeof (grammar_lines) / sizeof (grammar_lines
    [0]));
for (int i = 0; i < line_count; ++i)
{
    strncpy (lines[i], grammar_lines[i], MAX_LINE - 1);
    lines[i][MAX_LINE - 1] = '\0';
}

char nonterms[MAX_SYMBOLS][MAX_NAME];
char symbols[MAX_SYMBOLS][MAX_NAME];
Production prods[MAX_PRODS];
int nonterm_count = 0;
int symbol_count = 0;
int start_symbol = 0;
int prod_count = 0;

if (!parse_grammar_lines (lines, line_count, nonterms, &nonterm_count
    ,
        symbols, &symbol_count, &start_symbol,
        prods,
        &prod_count))
{
    fprintf (stderr, "Failed to parse grammar.\n");
    return 1;
}

char aug_name[MAX_NAME];
int max_base = (int)sizeof (aug_name) - 2;
snprintf (aug_name, sizeof (aug_name), "%.*s", max_base,
    symbols[start_symbol]);
int aug_sym = add_name (symbols, &symbol_count, aug_name);
if (aug_sym < 0)
{

```

```

    fprintf (stderr, "Too many symbols.\n");
    return 1;
}

Production all_prods[MAX_PRODS];
int all_prod_count = 0;
Production aug;
aug.lhs = aug_sym;
aug.rhs_len = 1;
aug.rhs[0] = start_symbol;
all_prods[all_prod_count++] = aug;
for (int i = 0; i < prod_count; ++i)
{
    all_prods[all_prod_count++] = prods[i];
}

int symbol_is_nonterm[MAX_SYMBOLS] = { 0 };
for (int i = 0; i < nonterm_count; ++i)
{
    int idx = name_index (symbols, symbol_count, nonterms[i]);
    if (idx >= 0)
    {
        symbol_is_nonterm[idx] = 1;
    }
}
symbol_is_nonterm[aug_sym] = 1;

int nonterm_first[MAX_SYMBOLS];
int nonterm_prod_count[MAX_SYMBOLS];
for (int i = 0; i < MAX_SYMBOLS; ++i)
{
    nonterm_first[i] = -1;
    nonterm_prod_count[i] = 0;
}
for (int i = 0; i < all_prod_count; ++i)
{
    int lhs = all_prods[i].lhs;
    if (nonterm_first[lhs] < 0)
    {
        nonterm_first[lhs] = i;
    }
}

```

```

        nonterm_prod_count[lhs]++;
    }

ItemSet states[MAX_STATES];
int state_count = 0;
int transitions[MAX_STATES][MAX_SYMBOLS];
for (int i = 0; i < MAX_STATES; ++i)
{
    for (int j = 0; j < MAX_SYMBOLS; ++j)
    {
        transitions[i][j] = -1;
    }
}

ItemSet start_set;
start_set.count = 0;
Item start_item;
start_item.prod = 0;
start_item.dot = 0;
itemset_add (&start_set, &start_item);
closure (&start_set, all_prods, nonterm_first, nonterm_prod_count,
        symbol_is_nonterm);
itemset_sort (&start_set);
states[state_count++] = start_set;

for (int i = 0; i < state_count; ++i)
{
    for (int sym = 0; sym < symbol_count; ++sym)
    {
        ItemSet next;
        goto_set (&states[i], sym, &next, all_prods, nonterm_first,
                nonterm_prod_count, symbol_is_nonterm);
        if (next.count == 0)
        {
            continue;
        }
        int existing = -1;
        for (int k = 0; k < state_count; ++k)
        {
            if (itemset_equal (&states[k], &next))
            {

```

```

        existing = k;
        break;
    }
}
if (existing < 0)
{
    if (state_count >= MAX_STATES)
    {
        fprintf (stderr, "Too many states.\n");
        return 1;
    }
    states[state_count] = next;
    existing = state_count;
    state_count++;
}
transitions[i][sym] = existing;
}
}

unsigned char first[MAX_SYMBOLS * MAX_SYMBOLS] = { 0 };
unsigned char follow[MAX_SYMBOLS * MAX_SYMBOLS] = { 0 };
unsigned char first_eps[MAX_SYMBOLS] = { 0 };
unsigned char follow_end[MAX_SYMBOLS] = { 0 };
compute_first (all_prods, all_prod_count, symbol_count,
               symbol_is_nonterm,
               first, first_eps);
compute_follow (all_prods, all_prod_count, symbol_count,
               symbol_is_nonterm,
               first, first_eps, follow, follow_end, aug_sym);

int term_count = 0;
int term_col[MAX_SYMBOLS];
for (int i = 0; i < MAX_SYMBOLS; ++i)
{
    term_col[i] = -1;
}
for (int i = 0; i < symbol_count; ++i)
{
    if (!symbol_is_nonterm[i])
    {
        term_col[i] = term_count;
    }
}

```

```

        term_count++;
    }
}

int nonterms_out[MAX_SYMBOLS];
int nonterm_out_count = 0;
for (int i = 0; i < symbol_count; ++i)
{
    if (symbol_is_nonterm[i] && i != aug_sym)
    {
        nonterms_out[nonterm_out_count++] = i;
    }
}

ActionCell action[MAX_STATES][MAX_SYMBOLS + 1];
int goto_table[MAX_STATES][MAX_SYMBOLS];
memset (action, 0, sizeof (action));
for (int i = 0; i < MAX_STATES; ++i)
{
    for (int j = 0; j < MAX_SYMBOLS; ++j)
    {
        goto_table[i][j] = -1;
    }
}

int conflict = 0;
for (int i = 0; i < state_count; ++i)
{
    for (int j = 0; j < states[i].count; ++j)
    {
        Item it = states[i].items[j];
        const Production *p = &all_prods[it.prod];
        if (it.dot < p->rhs_len)
        {
            int sym = p->rhs[it.dot];
            if (!symbol_is_nonterm[sym])
            {
                int to = transitions[i][sym];
                int col = term_col[sym];
                if (to >= 0 && col >= 0)
                {

```

```

        if (set_action (&action[i][col], ACT_SHIFT, to))
        {
            conflict = 1;
        }
    }
}
else
{
    if (it.prod == 0)
    {
        if (set_action (&action[i][term_count], ACT_ACCEPT,
            0))
        {
            conflict = 1;
        }
    }
    else
    {
        int lhs = p->lhs;
        for (int t = 0; t < symbol_count; ++t)
        {
            if (follow[lhs * MAX_SYMBOLS + t])
            {
                int col = term_col[t];
                if (col >= 0)
                {
                    if (set_action (&action[i][col],
                        ACT_REDUCE,
                            it.prod))
                    {
                        conflict = 1;
                    }
                }
            }
        }
        if (follow_end[lhs])
        {
            if (set_action (&action[i][term_count],
                ACT_REDUCE,
                    it.prod))

```

```

        {
            conflict = 1;
        }
    }
}

}

}

for (int j = 0; j < nonterm_out_count; ++j)
{
    int sym = nonterms_out[j];
    int to = transitions[i][sym];
    if (to >= 0)
    {
        goto_table[i][j] = to;
    }
}

if (conflict)
{
    fprintf (stderr, "SLR table has conflicts, parsing may fail.\n");
}

TokenItem tokens[MAX_TOKENS];
int token_count = load_tokens (token_path, tokens, MAX_TOKENS,
    symbols,
                                symbol_count, term_col);
if (token_count < 0)
{
    return 1;
}

int state_stack[MAX_TOKENS];
Attribute attr_stack[MAX_TOKENS];
int top = 0;
state_stack[top] = 0;

g_scope_level = 0;

int pos = 0;

```

```
ASTNode *root = NULL;

while (1)
{
    int state = state_stack[top];
    int lookahead_col = term_count;
    int lookahead_sym = -1;
    char lookahead_lexeme[MAX_NAME] = "";
    if (pos < token_count)
    {
        lookahead_sym = tokens[pos].symbol;
        lookahead_col = term_col[lookahead_sym];
        strncpy (lookahead_lexeme, tokens[pos].lexeme, MAX_NAME - 1);
        lookahead_lexeme[MAX_NAME - 1] = '\0';
    }

    ActionCell cell = action[state][lookahead_col];
    if (cell.type == ACT_SHIFT)
    {
        top++;
        state_stack[top] = cell.value;
        attr_stack[top].node = NULL;
        attr_stack[top].type = TYPE_UNKNOWN;
        attr_stack[top].lexeme[0] = '\0';

        if (lookahead_sym >= 0)
        {
            const char *sym_name = symbols[lookahead_sym];
            if (strcmp (sym_name, "id") == 0)
            {
                attr_stack[top].node = ast_create ("id",
                    lookahead_lexeme);
                strncpy (attr_stack[top].lexeme, lookahead_lexeme,
                    MAX_NAME - 1);
                attr_stack[top].lexeme[MAX_NAME - 1] = '\0';
            }
            else if (strcmp (sym_name, "num") == 0)
            {
                attr_stack[top].node = ast_create ("num",
                    lookahead_lexeme);
                strncpy (attr_stack[top].lexeme, lookahead_lexeme,
```



```

        MAX_NAME - 1);
        attr_stack[top].lexeme[MAX_NAME - 1] = '\\0';
    }
    else if (strcmp (sym_name, "rop") == 0)
    {
        strncpy (attr_stack[top].lexeme, lookahead_lexeme,
                MAX_NAME - 1);
        attr_stack[top].lexeme[MAX_NAME - 1] = '\\0';
    }
    else if (strcmp (sym_name, "{") == 0)
    {
        enter_scope ();
    }
    else if (strcmp (sym_name, "}") == 0)
    {
        leave_scope ();
    }
}

pos++;
}
else if (cell.type == ACT_REDUCE)
{
    int prod_id = cell.value;
    Production *p = &all_prods[prod_id];
    Attribute rhs_attrs[MAX_RHS];
    for (int i = p->rhs_len - 1; i >= 0; --i)
    {
        rhs_attrs[i] = attr_stack[top];
        top--;
    }

    Attribute result;
    result.node = NULL;
    result.type = TYPE_UNKNOWN;
    result.lexeme[0] = '\\0';

    switch (prod_id)
    {
        case 1: /* P -> Func */
            result.node = ast_create ("program", NULL);

```

```

    ast_add_child (result.node, rhs_attrs[0].node);
    break;
case 2: /* Func -> void id ( Params ) Block */
    result.node = ast_create ("func", rhs_attrs[1].lexeme);
    ast_add_child (result.node, rhs_attrs[3].node);
    ast_add_child (result.node, rhs_attrs[5].node);
    break;
case 3: /* Params -> ParamList */
    result = rhs_attrs[0];
    break;
case 4: /* Params -> epsilon */
    result.node = ast_create ("params", NULL);
    break;
case 5: /* ParamList -> ParamList , Param */
    result = rhs_attrs[0];
    ast_add_child (result.node, rhs_attrs[2].node);
    break;
case 6: /* ParamList -> Param */
    result.node = ast_create ("params", NULL);
    ast_add_child (result.node, rhs_attrs[0].node);
    break;
case 7: /* Param -> Type id */
    if (!declare_symbol (rhs_attrs[1].lexeme, rhs_attrs[0].
        type))
    {
        char msg[MAX_LINE];
        snprintf (msg, sizeof (msg), "Duplicate declaration
            of '%s'",
                rhs_attrs[1].lexeme);
        record_error (msg);
    }
    result.type = rhs_attrs[0].type;
    result.node
        = ast_create ("param", value_type_name (rhs_attrs[0].
            type));
    ast_add_child (result.node, rhs_attrs[1].node);
    break;
case 8: /* Type -> int */
    result.type = TYPE_INT;
    result.node = ast_create ("type", "int");
    break;

```

```
case 9: /* Type -> float */
    result.type = TYPE_FLOAT;
    result.node = ast_create ("type", "float");
    break;
case 10: /* Block -> { StmtList } */
    result.node = ast_create ("block", NULL);
    ast_add_child (result.node, rhs_attrs[1].node);
    break;
case 11: /* StmtList -> StmtList Stmt */
    result = rhs_attrs[0];
    ast_add_child (result.node, rhs_attrs[1].node);
    break;
case 12: /* StmtList -> Stmt */
    result.node = ast_create ("stmt_list", NULL);
    ast_add_child (result.node, rhs_attrs[0].node);
    break;
case 13: /* Stmt -> Decl */
case 14: /* Stmt -> Assign */
case 15: /* Stmt -> IfStmt */
case 16: /* Stmt -> WhileStmt */
case 17: /* Stmt -> ReturnStmt */
case 18: /* Stmt -> PrintStmt */
    result = rhs_attrs[0];
    break;
case 19: /* Decl -> Type DeclList ; */
    result.node
        = ast_create ("decl", value_type_name (rhs_attrs[0].
            type));
    ast_add_child (result.node, rhs_attrs[1].node);
    declare_decl_list (rhs_attrs[1].node, rhs_attrs[0].type);
    break;
case 20: /* DeclList -> DeclList , id */
    result = rhs_attrs[0];
    ast_add_child (result.node, rhs_attrs[2].node);
    break;
case 21: /* DeclList -> id */
    result.node = ast_create ("decl_list", NULL);
    ast_add_child (result.node, rhs_attrs[0].node);
    break;
case 22: /* Assign -> id = Expr ; */
    {
```

```

const char *name = rhs_attrs[0].lexeme;
ValueType lhs_type = TYPE_UNKNOWN;
if (!lookup_symbol (name, &lhs_type))
{
    char msg[MAX_LINE];
    snprintf (msg, sizeof (msg), "Undeclared identifier
        '%s'",
            name);
    record_error (msg);
    lhs_type = TYPE_ERROR;
}
if (lhs_type == TYPE_INT && rhs_attrs[2].type ==
    TYPE_FLOAT)
{
    char msg[MAX_LINE];
    snprintf (msg, sizeof (msg),
        "Type mismatch: cannot assign float to
            int '%s'",
            name);
    record_error (msg);
}
result.type = lhs_type;
result.node = ast_create ("assign", NULL);
ast_add_child (result.node, rhs_attrs[0].node);
ast_add_child (result.node, rhs_attrs[2].node);
break;
}

case 23: /* IfStmt -> if ( Cond ) Block else Block */
    result.node = ast_create ("if", NULL);
    ast_add_child (result.node, rhs_attrs[2].node);
    ast_add_child (result.node, rhs_attrs[4].node);
    ast_add_child (result.node, rhs_attrs[6].node);
    break;

case 24: /* WhileStmt -> while ( Cond ) Block */
    result.node = ast_create ("while", NULL);
    ast_add_child (result.node, rhs_attrs[2].node);
    ast_add_child (result.node, rhs_attrs[4].node);
    break;

case 25: /* ReturnStmt -> return ; */
    result.node = ast_create ("return", NULL);
    break;

```

```
case 26: /* PrintStmt -> print ( Expr ) ; */
    result.node = ast_create ("print", NULL);
    ast_add_child (result.node, rhs_attrs[2].node);
    break;
case 27: /* Cond -> Expr rop Expr */
    result.type = TYPE_INT;
    result.node = ast_create ("relop", rhs_attrs[1].lexeme);
    ast_add_child (result.node, rhs_attrs[0].node);
    ast_add_child (result.node, rhs_attrs[2].node);
    break;
case 28: /* Expr -> Expr + Term */
case 29: /* Expr -> Expr - Term */
    {
        ValueType lt = rhs_attrs[0].type;
        ValueType rt = rhs_attrs[2].type;
        if (lt == TYPE_ERROR || rt == TYPE_ERROR)
        {
            result.type = TYPE_ERROR;
        }
        else if (lt == TYPE_FLOAT || rt == TYPE_FLOAT)
        {
            result.type = TYPE_FLOAT;
        }
        else
        {
            result.type = TYPE_INT;
        }
        result.node = ast_create (prod_id == 28 ? "add" : "sub",
            NULL);
        ast_add_child (result.node, rhs_attrs[0].node);
        ast_add_child (result.node, rhs_attrs[2].node);
        break;
    }
case 30: /* Expr -> Term */
    result = rhs_attrs[0];
    break;
case 31: /* Term -> Term * Factor */
case 32: /* Term -> Term / Factor */
    {
        ValueType lt = rhs_attrs[0].type;
        ValueType rt = rhs_attrs[2].type;
```

```
if (lt == TYPE_ERROR || rt == TYPE_ERROR)
{
    result.type = TYPE_ERROR;
}
else if (lt == TYPE_FLOAT || rt == TYPE_FLOAT)
{
    result.type = TYPE_FLOAT;
}
else
{
    result.type = TYPE_INT;
}
result.node = ast_create (prod_id == 31 ? "mul" : "div"
    , NULL);
ast_add_child (result.node, rhs_attrs[0].node);
ast_add_child (result.node, rhs_attrs[2].node);
break;
}
case 33: /* Term -> Factor */
    result = rhs_attrs[0];
    break;
case 34: /* Factor -> ( Expr ) */
    result = rhs_attrs[1];
    break;
case 35: /* Factor -> id */
{
    ValueType id_type = TYPE_UNKNOWN;
    if (!lookup_symbol (rhs_attrs[0].lexeme, &id_type))
    {
        char msg[MAX_LINE];
        snprintf (msg, sizeof (msg), "Undeclared identifier
            '%s'",
                rhs_attrs[0].lexeme);
        record_error (msg);
        id_type = TYPE_ERROR;
    }
    result = rhs_attrs[0];
    result.type = id_type;
    break;
}
case 36: /* Factor -> num */
```

```

{
    result = rhs_attrs[0];
    if (strchr (rhs_attrs[0].lexeme, '.')
        || strchr (rhs_attrs[0].lexeme, 'e')
        || strchr (rhs_attrs[0].lexeme, 'E'))
    {
        result.type = TYPE_FLOAT;
    }
    else
    {
        result.type = TYPE_INT;
    }
    break;
}

case 37: /* Factor -> input ( ) */
    result.type = TYPE_INT;
    result.node = ast_create ("input", NULL);
    break;
default:
    result = rhs_attrs[0];
    break;
}

int next_state = state_stack[top];
int lhs = p->lhs;
int goto_state = -1;
for (int i = 0; i < nonterm_out_count; ++i)
{
    if (nonterms_out[i] == lhs)
    {
        goto_state = goto_table[next_state][i];
        break;
    }
}
if (goto_state < 0)
{
    fprintf (stderr, "Goto failed for nonterminal %s.\n",
             symbols[lhs]);
    return 1;
}
top++;

```

```

        state_stack[top] = goto_state;
        attr_stack[top] = result;
    }
    else if (cell.type == ACT_ACCEPT)
    {
        root = attr_stack[top].node;
        break;
    }
    else
    {
        fprintf (stderr, "Parse error at token %d.\n", pos + 1);
        return 1;
    }
}

printf ("AST:\n");
print_ast (root, 0);

printf ("\nSymbol Table:\n");
printf ("%16s %-8s %-8s\n", "Name", "Type", "Scope");
for (int i = 0; i < g_symbol_count; ++i)
{
    printf ("%16s %-8s %-8d\n", g_symbols[i].name,
            value_type_name (g_symbols[i].type), g_symbols[i].scope);
}

printf ("\nSemantic Errors:\n");
if (g_error_count == 0)
{
    printf ("None\n");
}
else
{
    for (int i = 0; i < g_error_count; ++i)
    {
        printf ("- %s\n", g_errors[i]);
    }
}

return 0;
}

```

A.2 运行命令

编译

```
gcc -Wall -Wextra -o semantic main.c
```

运行

```
./semantic /path/to/tokens.out
```